

PROJET D'ÉTUDES — RENDU INDIVIDUEL

ToolboxV8

Plateforme d'automatisation de tests d'intrusion

Auteur	Titouan AKA A MFOULA
Rôle principal	Interface & Reporting — Frontend Jinja2, UI chips, générateur de rapports PDF/HTML/CSV
Promotion	Mastère Cybersécurité 2025/2026
École	SUP DE VINCI
Date	Juin 2026

1. Perspectives d'évolution

1.1 Un workflow guidé entre les modules

Aujourd'hui, c'est à l'analyste de décider quel module lancer après un scan. Ce serait intéressant que l'interface fasse une partie de ce travail : si le module recon détecte un port 22 ouvert, elle pourrait suggérer automatiquement un profil Hydra SSH, cible déjà pré-remplie. Ce type de logique conditionnelle est faisable avec notre stack Jinja2 + JavaScript, ça demanderait surtout de formaliser les règles de transition entre modules.

1.2 Des rapports personnalisables par client

Le rapport PDF qu'on génère est professionnel et lisible, mais il a un seul format. Pour un cabinet qui audite les mêmes clients régulièrement, ce serait utile de pouvoir sauvegarder des templates : logo du client, palette de couleurs, niveau de détail selon le public visé (synthèse CODIR vs technique complète pour l'équipe SOC). ReportLab le permet nativement, il faudrait juste exposer ces options dans l'interface et les passer au générateur.

1.3 Un module de classification automatique des vulnérabilités

ZAP remonte 105 alertes triées par sévérité OWASP, mais c'est encore à l'analyste de juger ce qui est vraiment prioritaire dans le contexte du client. Un modèle de classification entraîné sur des données CVE pourrait attribuer un score contextualisé, en tenant compte du type d'application audité. Ça s'intégrerait dans le pipeline Celery existant, en post-traitement du scan.

1.4 Un mode campagne pour le suivi dans le temps

Actuellement, chaque scan est un événement isolé. Un mode campagne permettrait de regrouper plusieurs scans sous un même dossier client, de suivre l'évolution des vulnérabilités dans le temps — première détection, retest, fermeture — et de générer un rapport de synthèse. Ça ferait passer ToolboxV8 d'un outil ponctuel à une vraie plateforme de suivi continu, ce qui correspond mieux aux besoins réels d'un cabinet offensif.

1.5 Internationalisation

L'interface est entièrement en français, ce qui colle au contexte client actuel. Mais si la toolbox est amenée à être distribuée plus largement, une internationalisation via Flask-Babel ou équivalent s'imposerait. Ce n'est pas une priorité immédiate, mais autant garder ça en tête pour ne pas avoir à tout refactorer plus tard.

2. Analyse critique — limites et retours d'expérience

2.1 ReportLab, plus complexe que prévu

Le générateur de rapports a été de loin la partie la plus chronophage de mon périmètre. J'avais sous-estimé les subtilités de ReportLab : les styles ne se propagent pas sur les pages d'overflow, les couleurs de fond doivent être re-déclarées à chaque frame, et les blocs de code longs pouvaient dépasser la page sans être tronqués proprement. J'ai dû itérer plusieurs fois, notamment pour corriger le bug texte blanc sur fond blanc qui rendait les pages 2+ illisibles. Rétrospectivement, j'aurais dû prototyper le rapport sur de vraies données dès le début plutôt que sur des données fictives — le problème serait apparu beaucoup plus tôt.

2.2 Le choix JavaScript vanilla — assumé mais limitant

J'ai volontairement évité d'introduire un framework JS pour rester cohérent avec Jinja2 et ne pas complexifier le build. C'est défendable pour un MVP. Mais quand il a fallu gérer la modale wordlist, le catalogue de dorks avec sélection multiple et le polling temps réel en parallèle, ça a donné du JavaScript procédural assez dense, difficile à tester et à maintenir. Pour une V2, je regarderais du côté d'Htmx ou Alpine.js — légers, compatibles Jinja2, et ils auraient évité pas mal de ce code.

2.3 Aucun test frontend

Le backend a des tests pytest avec PostgreSQL et Redis réels. Le frontend, lui, n'a aucun test automatisé. Les chips, la validation de cible et le polling ont tous été validés à la main. Quand j'ai dû appliquer le bypass de validation pour `passive_recon` et Hydra, j'ai dû le faire à plusieurs endroits parce que la logique était dupliquée entre le front et le back, et je l'ai découvert en testant manuellement. Des tests Playwright auraient capturé ça bien plus tôt.

2.4 Le piège du rebuild worker

Le générateur PDF tourne dans le worker Celery, pas dans l'API. Pendant un moment, je modifiais le générateur et je rebuillais uniquement l'image API sans voir d'effet — logique, mais pas du tout évident quand on débogue sous pression. Ce n'est pas une mauvaise architecture en soi, c'est même cohérent que les scans et les rapports partagent le même contexte d'exécution. Mais j'aurais dû le documenter explicitement dès le départ pour éviter de perdre du temps dessus.

2.5 La cohérence visuelle PDF / HTML

Les deux formats de rapport ont parfois divergé en cours de projet. Quand j'ai corrigé la palette du PDF, j'ai dû synchroniser manuellement le template HTML. Avec un système de design partagé défini dès le départ — une palette commune, des variables CSS et des constantes ReportLab liées — ça n'aurait pas été un problème.

3. Analyse personnelle

3.1 Ce que ce projet m'a vraiment appris

Mon rôle principal était l'interface et le reporting, mais comme on l'a dit en présentation, on a tous mis les mains dans plusieurs parties du projet — c'était important pour la cohérence du produit et pour que chacun puisse en parler en soutenance. De mon côté, j'ai contribué à l'intégration de certains modules et j'ai bossé sur la gestion de la base de données. Rien de comparable au travail d'Ayoub sur l'architecture back-end ou d'Abdallah sur les outils offensifs, mais ça m'a permis de comprendre le projet dans son ensemble plutôt que de rester dans ma bulle frontend.

J'avais déjà une expérience avec PostgreSQL depuis ma L3 — gestion de schémas, requêtes, un peu d'ORM — donc cette partie ne m'était pas étrangère. Par contre, le frontend dynamique, c'était clairement nouveau pour moi. J'avais fait du CSS et du HTML statique, mais pas de pages avec état, polling asynchrone, ou gestion de formulaires complexes comme les chips ou les modales. J'ai dû monter en compétence assez vite sur JavaScript et sur Jinja2.

3.2 L'utilisation de l'IA dans le projet

J'ai utilisé des outils d'IA — principalement comme assistant de développement — sur trois axes : génération de code pour les parties répétitives ou les boilerplates que je connaissais mal (notamment les structures JS pour le polling), aide au débogage quand je bloquais sur un comportement inattendu de ReportLab, et relecture de documentation technique. Ce que j'ai appris, c'est que l'IA est utile pour aller plus vite sur ce qu'on comprend déjà, mais qu'elle peut induire en erreur sur ce qu'on maîtrise mal. Plusieurs fois, elle m'a proposé des solutions qui semblaient correctes mais qui ne tenaient pas compte des contraintes spécifiques de notre architecture — le coup du rebuild worker, par exemple, un outil IA ne pouvait pas le deviner sans contexte. Ça m'a confirmé qu'elle s'utilise comme un accélérateur, pas comme un remplaçant de la compréhension.

3.3 Forces

- Sens de l'UX : supprimer les textareas, passer aux chips, adapter la validation selon le module — ces décisions viennent d'une capacité à me mettre à la place d'un analyste qui ne veut pas apprendre la syntaxe CLI de chaque outil.
- Rigueur sur le rendu final : j'aurais pu livrer un rapport PDF "qui marche" bien plus tôt. J'ai préféré itérer jusqu'à un rendu professionnel que le client peut envoyer directement à ses propres clients.
- Initiative : le catalogue de Google Dorks pour `passive_recon` n'était pas dans le périmètre initial. Je l'ai proposé, conçu et intégré, et c'est devenu l'une des fonctionnalités les plus utilisées en démo.
- Communication dans l'équipe : être à l'interface entre le backend et les modules offensifs demandait de comprendre les deux côtés, et j'ai trouvé ça naturel.

3.4 Faiblesses

- Mauvaise estimation du reporting : j'ai passé trop de temps sur les maquettes d'interface avant d'attaquer ReportLab, ce qui a compressé le temps disponible pour l'itérer correctement.
- Tendance à trop finir avant de livrer : la modale wordlist est complète et bien pensée, mais j'aurais pu livrer une version minimale bien plus tôt et l'améliorer ensuite.
- Connaissance insuffisante de ReportLab en début de projet : j'aurais gagné beaucoup de temps en lisant la doc complète plutôt qu'en découvrant ses comportements par l'expérimentation.

3.5 Compétences développées

- Jinja2 pour des templates HTML dynamiques complexes — modales, chips, validation adaptative, polling.
- Génération de PDF programmatiques avec ReportLab : gestion des styles, pages d'overflow, mise en page professionnelle.
- JavaScript asynchrone sans framework : fetch API, gestion d'états, polling Celery.
- Compréhension du cycle complet d'un pentest : construire l'interface de chaque module m'a forcé à comprendre ce que fait chaque outil offensif, ce qui m'a apporté une vraie culture sécurité.
- Utilisation raisonnée de l'IA comme outil de développement, avec ses avantages et ses limites.
- Communication technique orale : préparer le script de présentation et structurer les slides m'a appris à vulgariser des choix techniques complexes.

3.6 Pour les prochains projets

- Prototyper sur des données réelles dès le départ — ne jamais développer un composant de rendu sans l'alimenter avec de vraies sorties.
- Écrire des tests frontend dès le début, même simples.
- Définir un système de design partagé avant de coder : une source de vérité unique pour éviter les divergences entre formats de sortie.
- Time-boxer les phases d'exploration : fixer une limite de temps pour la recherche et passer à une solution de fallback si elle n'est pas atteinte.
- Mieux documenter les décisions d'architecture au fil de l'eau, pas seulement le code.

Document rédigé dans le cadre du Projet d'Études — Mastère Cybersécurité 2025/2026 — SUP DE VINCI
